

Guía para la entrega de proyectos

(c) [Diego Fernando Marín Lozano](#), ver 2026-09-01

Introducción

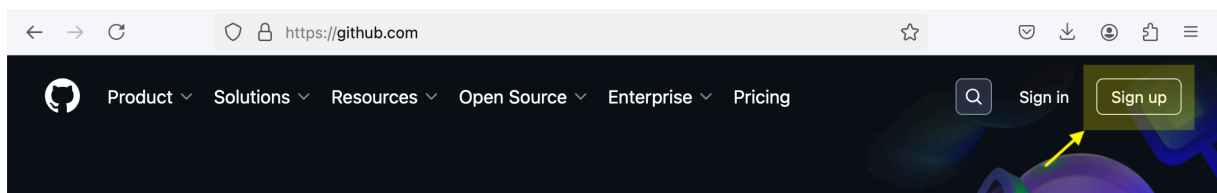
[Git](#) es un sistema de control de versiones, un programa capaz de rastrear los cambios en el código fuente (y otros archivos de texto) de un proyecto de software. **Git** se ha convertido en una herramienta esencial para los desarrolladores de software, pues permite que equipos dispersos trabajen en forma colaborativa y eficiente.

[GitHub](#) por otro lado, es una plataforma web que ofrece espacio de almacenamiento remoto para repositorios de **Git**. Del mismo modo que lo hace [GitLab](#), [Bitbucket](#) y [OneDev](#) entre otros.

Creación de la Cuenta de GitHub

Lo primero que debes hacer es [crear tu cuenta personal](#) en **GitHub**, sigue estos pasos:

- Ingresa a <https://github.com>
- Haz click en **Registrarse**:



- Sigue las indicaciones de la página para crear tu cuenta personal.

Nota: Es muy importante validar tu correo electrónico.

- Realiza la [configuración de tu perfil](#), será tu carta de presentación como desarrollador.
- Configura la [llave SSH](#) para tu conexión al repositorio remoto.

Instalación

Linux

- Bien sea que tengas Linux o [WSL](#) en Windows (por defecto es [Ubuntu](#)), puedes instalar `git` con estos comandos, según sea el Linux que utilices:

```
# Debian, Ubuntu y derivados
sudo apt install git
# Red Hat, y derivados
sudo dnf install git
```

MacOS

- En MacOS puedes utilizar [Homebrew](#) para instalar `git` :

```
brew install git
```

Windows

- Utiliza el instalador oficial de [git](#) para windows (es el mismo de [git-for-windows](#)). Esto instala el comando `git` , junto con `bash` y otros comandos útiles para desarrollo.

Configuración

- Configura `git` con tus datos:

```
git config --global user.name "Juan Perez"
git config --global user.email "juanperez@correo.com"
```

Entrega de Trabajos

El proceso de preparación y entrega un trabajo a través de GitHub es el siguiente:

- Ingresa a GitHub y [crea el repositorio del proyecto](#).
- En tu computador, crea el directorio del proyecto, inicializa el repositorio y vincúlalo a

GitHub:

```
mkdir proyecto
cd proyecto
git init
git remote add origin proyecto_repo
```

- Inicia Visual Studio Code (conectado a WSL):

```
code .
```

- Añade el archivo `.gitignore` al proyecto con el contenido generado. Utiliza gitignore.io para crear el contenido del archivo `.gitignore` en forma automática.
- Añade el archivo `README.md` al proyecto con el contenido generado. Utiliza readme.so para crear el archivo `README.md` con un contenido estándar.
- Crea el entorno virtual y activarlo:

```
python3 -m venv venv
source venv/bin/activate
```

- Instala las dependencias que tu Proyecto necesita, y regístralas en el archivo `requirements.txt`:

```
pip install flask dotenv rich pydantic
pip freeze > requirements.txt
```

- Añade los archivos con el código, y agrégalos al repositorio (al **Stage Are**):

```
git add .
```

- Crea un `commit` por cada modificación importante del proyecto (al **Repositorio Local**):

```
git commit -m "explicación del cambio realizado"
```

- Con los cambios importantes, regístralos en **GitHub** (al **Repositorio Remoto**):

```
git push origin main
```

- Repite, hasta terminar el proyecto.
- Desactiva el entorno virtual, antes de pasar a otro proyecto:

```
deactivate
```

Comparte con el profesor, el **URL** del proyecto en tu cuenta de **GitHub**:

```
https://github.com/estudiante/proyecto
```

Cómo el profesor va a evaluar el proyecto?

- Clona el proyecto, desde el repositorio en **GitHub**:

```
git clone https://github.com/estudiante/proyecto apellido
```

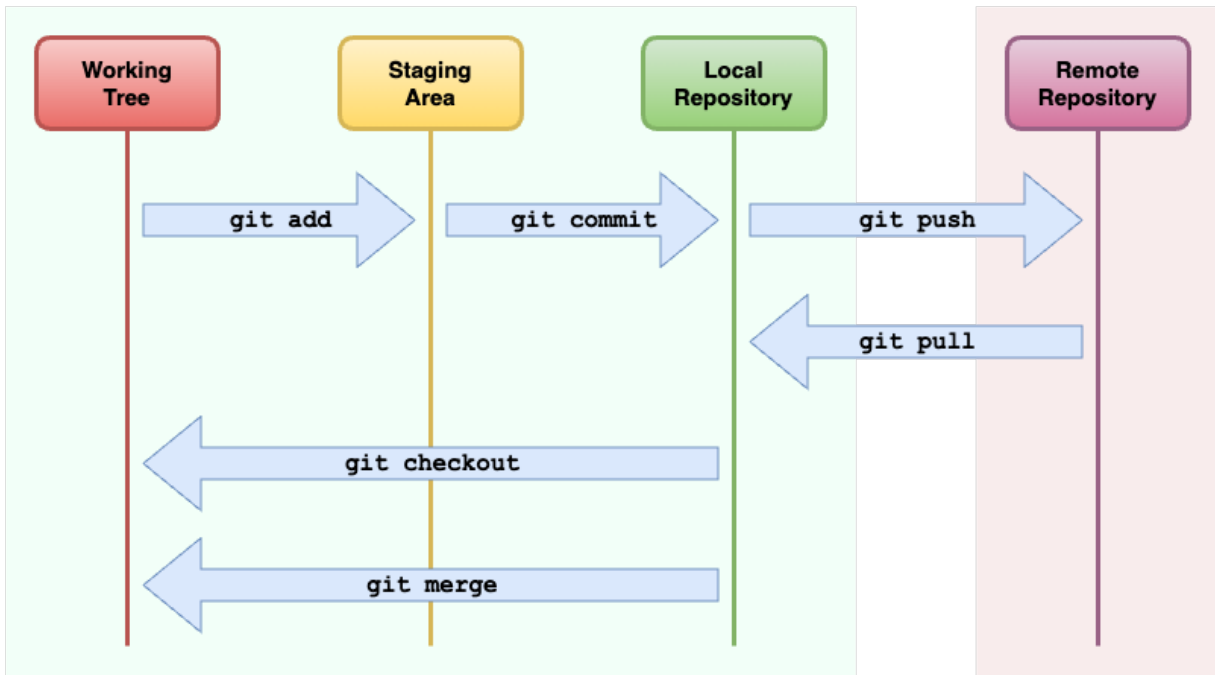
- Ingresa al directorio, configura, activa el entorno virtual e instala las dependencias:

```
cd apellido
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

- Ejecuta el proyecto, según las indicaciones del `README.md` : Si la forma de ejecutar el proyecto es diferente, los pasos completos y detallados deben estar en el `README.md` .

```
python3 app/main.py
```

Glosario



- **Repositorio local** es un directorio en tu computador que contiene la copia completa del proyecto, con todo el código fuente y demás archivos, junto con los metadatos de **Git** que registran el historial de cambio.
- **Repositorio remoto** es una copia de tu repositorio almacenada en un servidor **Git** remoto, que puede ser compartida y actualizada con otras personas. Este repositorio remoto suele llamarse `origin`.
- **Working Tree** es el directorio que utilizas para trabajar, donde modificas el código del proyecto y los demás archivos según se necesite. Estos archivos pueden estar en diferentes estados: sin modificar, modificados o nuevos.
- **Stage Area** (área de preparación) es una zona intermedia temporal donde se preparan los cambios que se van a incluir en el próximo commit.
- **Commit** (confirmación) es un punto de control en el historial del proyecto, que guarda un conjunto de cambios y su mensaje descriptivo, en un momento específico. Es como una foto instantánea que registra los cambios realizados desde el anterior commit, y en conjunto crean el historial de cambios.
- **HEAD** el commit más reciente en la rama activa y sirve como un puntero de referencia dentro del repositorio.
- **Branch** (rama) es una bifurcación o flujo de trabajo independientes de la línea de

desarrollo **main** (antes llamada **master**), permite al equipo realizar mejoras o correcciones simultáneas, en un mismo proyecto.

- **Merge** (fusión) es el proceso de combinar los cambios de una rama en otra, y se utiliza para integrar el trabajo realizado en diferentes ramas.
- **Checkout** es el proceso de cambiar entre ramas o a un commit específico, actualizando el estado de tu working tree al de la rama o commit seleccionado.
- **Tags** es una etiquetas que se asigna a un commit específico dentro del historial de cambios, marcando hitos importantes en el desarrollo de un proyecto, como lanzamientos, versiones o parches.
- **Pull request** (PR) es una solicitud para fusionar los cambios de una rama en otra en un repositorio remoto, lo que permite a los colaboradores revisar y discutir los cambios antes de realizar la fusión.
- **Clone** (clonar) es el proceso de copiar un repositorio remoto a tu máquina local, lo que incluye todo el historial del proyecto, las ramas, commits, etc.
- **Push** es la acción de enviar los commits realizados en tu repositorio local al repositorio remoto, actualizando el estado del proyecto compartido.
- **Pull** es la descarga y fusión los cambios más recientes del repositorio remoto a tu repositorio local, actualizando el estado del proyecto local con los últimos commits del remoto.
- **Fetch** es descargar los cambios del repositorio remoto sin fusionarlos en tu rama local. Te permite ver los cambios disponibles antes de decidir si se aplican.
- **Rebase** es la acción de reorganiza los commits de una rama, aplicando los cambios uno por uno sobre otro punto del historial. Es una alternativa al merge, pero modifica el historial de commits.
- **Conflict** (conflicto) es cuando **Git** no puede fusionar automáticamente los cambios de diferentes ramas debido a modificaciones contradictorias en los mismos archivos, y requiere una resolución manual.
- **Fork** (bifurcar) es la acción de crear una copia de un repositorio en tu cuenta personal, generalmente para hacer contribuciones a un proyecto de código abierto sin afectar el repositorio original.
- **Blame** es una herramienta de **Git** que muestra qué cambios y qué usuarios modificaron

cada línea de un archivo, útil para rastrear el origen de errores o cambios.

- **Stash** es la acción de guardar temporalmente los cambios que tienes en el working tree y el stage area sin hacer un commit, permitiéndote volver a un estado limpio antes de realizar otros cambios o tareas.
- **Cherry-pick** es el proceso de aplicar un commit específico de una rama a otra, sin fusionar toda la rama.
- **Gitignore** es un archivo donde se define qué archivos o directorios deben ser ignorados por **Git**, es decir: no seguir, ni añadir al repositorio.
- **State** cada archivo en el Working Tree tiene un estado:
 - **untracked** es un archivo no rastreado por **Git**, significa que nunca ha sido adicionado al stage area y por lo tanto tampoco paso por un commit.
 - **tracked** es un archivo que ya pasó a un commit pero no está en el stage area.
 - **staged** es un archivo en el stage area listo para el próximo commit.
 - **dirty / modified** es un archivo modificado, pero esos cambios no están en el stage area.